SwiftDrop Parcel Delivery & Courier Management System

Engineering Rules, Development Standards & Best Practices

Version 1.0 | Proposed team baseline | 21 October 2026

How this document relates to the other project documents

Use this alongside the SwiftDrop SRS and Team Work Division document. The SRS describes what the system must do; the work-division document identifies who owns each area; this document describes how the team should design, implement, test, secure, and maintain that work.

1. Purpose, scope, and authority

This document defines the minimum engineering rules, design principles, development practices, security controls, and team workflow for SwiftDrop. It is intended to keep ten members' contributions compatible and to make the system understandable, testable, secure, and reproducible.

This is the team's proposed engineering baseline, not a claim that every item is mandated by the assignment. The team should approve any project-specific choices before implementation. If this document conflicts with the assignment rubric or the approved Software Requirements Specification (SRS), the assignment and approved SRS take precedence.

- Must / MUST: required for the project unless the team records an approved exception.
- Should / SHOULD: strongly recommended; deviations need a reason recorded in the decision log.
- May / MAY: optional depending on scope, time, and demonstrated need.

2. Architecture and design rules

2.1 Service boundaries

- Use a Go modular monolith for core business capabilities: identity/access, parcel management, pricing, dispatch/couriers, proof/delivery records, merchant integration, and admin-facing business operations.
- Keep the Tracking API as a separately deployable service so read-heavy tracking can scale independently. It reads current authoritative parcel/status data and must not invent a separate conflicting source of truth.
- Use a separate Outbox Relay and Event Worker for asynchronous publication and consumption. Keep responsibilities distinct: domain modules write outbox records; relay publishes; workers perform notifications and webhook delivery.
- Keep the AI Assistant in a separate service. It calls approved backend APIs/tools and must not connect directly to the application database.
- Do not split the Core API into more microservices without a documented reason, clear data ownership, and a team-approved design decision.
- Use an API Gateway as the unified edge proxy for external routing, rate limiting, and client access control. Retain PostgreSQL and RabbitMQ as the foundational persistence and asynchronous transport layers supporting all modular and independent services.

2.2 Domain-Driven Design (DDD) rationale and bounded contexts

- Adopt Domain-Driven Design (DDD) principles selectively to manage business logic complexity and establish explicit module boundaries, avoiding full or heavyweight DDD overhead.
- Synthesize strategic DDD with a Go modular monolith for the Core API, deploy separate services only where justified by independent scaling or specialized processing (Tracking API, Outbox Relay, Event Worker, AI Assistant), and enforce Clean/Hexagonal Architecture boundaries within modules.
- Structure business capabilities into bounded contexts so each of the 10 team members can independently own an assigned business capability without fragmenting the system into microservice operational sprawl.

- Encapsulate interconnected business rules within domain modules rather than scattering logic across HTTP transport handlers, SQL queries, or frontend clients. Specific business lifecycles requiring domain-level protection include:
- Parcels progressing through a strictly controlled state machine and delivery lifecycle.
- Dispatchers assigning and reassigning parcels to couriers under route and capacity constraints.
- Couriers recording delivery attempts, validating proof-of-delivery attachments, and registering failure classifications.
- Dynamic pricing calculations, dimension evaluations, and fee determination rules.
- Merchant parcel creation, tenant isolation, and outbound status webhook dispatch.
- Customer access boundaries restricting visibility strictly to authorized personal shipments.
- Reliable event publication guaranteed through atomic transaction commits.

2.3 Selective tactical DDD pattern standards

- Apply tactical DDD patterns selectively as targeted architectural tools rather than a rigid checklist; introduce patterns only where business complexity justifies formal encapsulation.
- Ubiquitous Language: Agree on unambiguous domain terminology (such as Parcel, Delivery Attempt, Assignment, Tracking Event, Courier, and Merchant) and enforce its uniform usage across Go source code, database tables, API contracts, and team documentation.
- Bounded Contexts: Define strict logical boundaries around primary business domains: Identity & Access, Parcel Management, Dispatch & Couriers, Pricing, Merchant Integration, and Delivery Records.
- Entities: Model domain objects that possess distinct identity, attribute mutability, and an operational lifecycle (such as Parcel, Merchant, DeliveryAttempt, and Courier).
- Value Objects: Model immutable domain concepts defined exclusively by their values (such as Money, Address, ParcelDimensions, and TrackingNumber). Value objects must be self-validating upon construction and contain no independent identity.
- Aggregates and Aggregate Roots: Enforce business consistency invariants through aggregate roots. The aggregate root (such as Parcel) serves as the sole external gatekeeper for mutating internal state and controlling status transition rules. Keep aggregate boundaries small to prevent locking conflicts and excessive operational coupling.
- Domain Services: Encapsulate stateless domain logic or cross-entity operations (such as dynamic fee calculations and route feasibility checks) that do not naturally belong within a single entity.
- Domain Events: Represent significant, immutable business facts (such as ParcelBooked, CourierAssigned, DeliveryAttemptRecorded, and DeliveryCompleted).
- Repositories: Expose clean persistence interfaces within the domain and application layers, decoupling aggregate storage and retrieval completely from SQL drivers and schema implementations.

2.4 Transactional state transition and invariant execution flow

- Prohibit direct database mutations: Do not execute raw, unvalidated SQL updates against parcel, assignment, or delivery status columns (for example, raw UPDATE parcels SET status = 'DELIVERED'). Raw mutations bypass domain invariants, courier assignment validation, proof-of-delivery checks, and legal state transitions.
- Enforce a strict layered execution flow for all domain state modifications (illustrated by a courier marking a parcel as delivered):
- 1. Transport Layer (HTTP Handler): Validates incoming request syntax and authentication context, extracts courier credentials, and maps parameters into an application command DTO.
- 2. Application Layer (Use Case Service): Coordinates the transaction boundary, retrieves the target Parcel aggregate from the repository, and delegates operation execution to the domain aggregate.
- 3. Domain Layer (Aggregate Root Invariants): The Parcel aggregate verifies courier assignment validity, confirms the parcel is in transit, validates attached proof-of-delivery records, and executes the state transition.
- 4. Persistence Layer (Single PostgreSQL Transaction): Atomically persists the updated parcel aggregate state, inserts an immutable audit record into parcel_status_history, and writes an integration event into the outbox table within the same database transaction.
- 5. Asynchronous Outbox Relay to RabbitMQ: The Outbox Relay process reads committed outbox entries and publishes events to RabbitMQ; downstream notification workers and merchant webhook handlers process events asynchronously.
- Transactional integrity and idempotency: Domain state updates, audit history trails, and outbox event records must commit atomically in a single database transaction. The relay publishes only post-commit, and all downstream consumers must implement idempotency to tolerate duplicate delivery.

2.5 Core API module structure and clean dependency rules

- Organize the Go Core API as a modular monolith adhering to Clean/Hexagonal Architecture principles with the following canonical directory layout:
- cmd/api/main.go: Application entrypoint; parses runtime configuration and delegates setup to the wiring container.
- internal/identity/: Authentication, credentials, JWT issuance, user identities, and role-based access control.
- internal/parcels/: Parcel aggregate root, parcel booking, lifecycle state transitions, and status history persistence.
- internal/pricing/: Fee calculation domain services, package size evaluation, and distance tariff engines.
- internal/dispatch/: Courier assignment use cases, dispatch board management, and capacity validation.
- internal/merchants/: Merchant accounts, API key validation, and webhook subscription models.
- internal/delivery/: Delivery attempt recording, proof-of-delivery photos/signatures, and failure reason classification.
- internal/admin/: Administrative operations: delivery zone configuration, system settings, cross-domain audit queries, and operational reports.
- internal/events/: Event and integration support: canonical event envelope definitions, transactional outbox writer interfaces, and schema serialization helpers.
- internal/platform/: Reusable cross-cutting technical drivers (PostgreSQL database pool, authentication middleware, RabbitMQ publisher).
- internal/app/wiring.go: Centralized dependency injection container; constructs repositories, injects services, initializes HTTP handlers, and configures the HTTP router.
- Enforce strict unidirectional dependency rules across internal module layers:
- Domain Layer (domain/): Contains pure business entities, value objects, domain errors, and aggregate invariants. Must have zero external dependencies; strictly forbidden from importing HTTP, SQL, RabbitMQ, or framework packages.
- Application Layer (application/): Defines use cases, orchestrates transactions, handles application commands/queries, and declares repository interfaces. Depends solely on the domain layer.
- Infrastructure Layer (infrastructure/): Implements application repository interfaces using concrete PostgreSQL queries, message publishers, and third-party integrations.
- Transport Layer (transport/): Houses HTTP handlers, routing, request parsing, input validation, status code mapping, and DTO serialization. Maps HTTP requests into application commands and calls use cases.
- Avoid structural overhead: Adopt full four-layer subpackaging only where justified by business complexity. Do not create redundant empty layer folders for straightforward CRUD operations or basic utility modules.

2.6 Capability ownership across the 10-member team

- Preserve the established 10-member work division while embedding explicit Domain-Driven Design responsibilities within each member's area of ownership:
- Members 1-4 (Core Domain Capabilities): Own the domain logic, entities, value objects, aggregates, and application use cases for assigned capabilities (Identity & Access, Parcel Management, Dispatch & Couriers, Pricing, and Merchant Integration).
- Member 5 (Tracking API Service): Implements tracking as an independent, read-oriented projection service; reads authoritative parcel status without defining or mutating parcel lifecycle business rules.
- Members 6-7 (Customer, Merchant, and Courier Frontends): Consume published API contracts; frontend components present data and capture input but must never evaluate or enforce domain business rules.
- Member 8 (Messaging, Outbox Relay & Event Consumers): Owns outbox publisher plumbing, broker infrastructure, and event consumer frameworks; domain module owners retain authority over which business events their modules generate.
- Member 9 (DevOps, Infrastructure & Observability): Owns deployment manifests, CI/CD pipelines, container clustering, and observability stacks without embedding domain logic into deployment configurations.
- Member 10 (AI Assistant Service): Implements specialized AI assistant tools that interface exclusively with approved application APIs; the assistant must never access the database directly or implement a secondary set of business rules.
- Authoritative lifecycle consensus: The team MUST formally agree upon and document the parcel lifecycle states, allowed transition matrices, and authoritative persistence ownership before parallel feature implementation begins, preventing divergent status interpretations across Parcel Management, Dispatch, and Tracking.

2.7 Single source of truth and state transitions

- PostgreSQL is authoritative for business and parcel state. RabbitMQ is message transport, not the authoritative parcel database. Redis, if used, is a cache/fan-out mechanism only.
- Define one canonical parcel lifecycle and enforce allowed transitions on the backend. Clients must never choose arbitrary status strings or bypass transition rules.

- Every status change must be auditable with parcel ID, previous/new state where appropriate, actor, timestamp, and relevant note or failure reason.
- Represent delivery attempts separately when required so failed attempts remain visible and redelivery follows explicit rules.
- Use a transaction for related business changes, status history, and outbox event creation. Do not publish an event before the corresponding business transaction commits.

2.8 DDD complexity guardrails and anti-patterns to avoid

- Apply DDD selectively as a reasoning tool rather than dogmatically; overengineering domain abstractions slows delivery and adds no architectural value.
- Do not decompose every bounded context into an independent microservice; maintain the Go modular monolith for all core business domains.
- Do not model every database table as an aggregate root; reserve aggregates strictly for transactional boundaries enforcing business invariants.
- Do not introduce redundant interfaces, repositories, or wrapper layers where a single concrete PostgreSQL implementation suffices and no isolation boundary is needed.
- Do not introduce CQRS or Event Sourcing frameworks prematurely; retain PostgreSQL relational tables as the single authoritative source of truth.
- Do not route intra-monolith module communications through RabbitMQ; execute synchronous internal operations via standard Go in-process calls or domain events.
- Do not allow multiple modules or services to mutate parcel status independently; all state transitions must be guarded and executed exclusively by the authoritative Parcel aggregate.

2.9 Keep complexity justified

- Prefer the simplest design that satisfies the SRS and demo requirements. Avoid adding services, databases, caches, abstractions, or frameworks without a specific benefit.
- Document important architecture decisions (for example gateway, token strategy, SSE fan-out, failed-delivery semantics) in short Architecture Decision Records (ADRs).
- Use synchronous REST for immediate request/response work and asynchronous messaging when the work can be retried or should not block the user request.
- Make failure behavior and ownership explicit: every service, queue, API, database table, and scheduled job should have a clear owner.

3. Repository, Git, and collaboration rules

3.1 Repository structure

- Use a consistent repository layout agreed by the team. Keep each service's code, tests, Dockerfile, and README in predictable locations.
- Store API specifications, event schemas, architecture diagrams, migration files, environment examples, and test reports in version control.
- Do not commit generated binaries, local databases, logs, credentials, real user data, or large build artifacts unless explicitly required.

3.2 Branches and commits

- Use short-lived feature branches; do not commit directly to the protected main branch.
- Use clear commit messages, such as `feat(tracking): add SSE endpoint`, `fix(auth): reject expired tokens`, or `test(dispatch): cover assignment conflict`.
- Keep pull requests focused and small enough to review. Avoid mixing unrelated refactors and feature work in one PR.
- Before opening a PR, pull/rebase the latest shared branch, run formatter/linter/tests, and check for accidental secrets or unrelated changes.
- Use issue or task IDs when the team has a tracker. Every task should have an owner, acceptance criteria, and status.

3.3 Pull-request reviews

- Every PR requires at least one teammate review; author should respond to comments and confirm fixes.

- Changes to authentication, authorization, migrations, event contracts, secrets, gateway rules, or Kubernetes exposure should receive careful review from a relevant owner.
- Review correctness, security, test coverage, error handling, maintainability, and documentation—not only whether the code runs.
- Do not approve code you have not read or cannot explain. Do not merge with failing required checks without an explicitly recorded decision.

4. General coding standards

4.1 All languages

- Use descriptive names and small functions with one clear purpose. Prefer readable code over clever shortcuts.
- Validate untrusted inputs at system boundaries. Keep validation close to the API boundary and enforce business rules in backend domain logic.
- Return structured errors; do not expose stack traces, SQL details, tokens, secrets, or internal infrastructure to clients.
- Use context-aware cancellation, timeouts, and bounded retries for network and database calls.
- Do not silently swallow errors. Log enough context to investigate while avoiding personal data and secrets.
- Remove dead code, debug prints, commented-out implementations, and unexplained TODOs before merging.
- Use dependency versions that are pinned or reproducibly managed; review new dependencies for maintenance, license, and security implications.

4.2 Go backend

- Run `gofmt` on all Go files and use the team's agreed linting checks.
- Keep handlers thin: parse and validate HTTP input, call application/domain logic, map errors to HTTP responses, and return DTOs.
- Separate transport DTOs from persistence models where doing so protects API stability or avoids leaking internal fields.
- Pass `context.Context` to request-scoped operations and set timeouts for outbound HTTP, broker, and database calls.
- Use dependency injection through constructors/interfaces where it improves testing; do not create interfaces for every concrete type without a reason.
- Never use `panic` for expected request errors. Handle errors explicitly and wrap them with useful context.
- Use parameterized SQL or a safe ORM query API; never concatenate untrusted values into SQL.
- Expose separate liveness and readiness behavior where appropriate; readiness should fail if the service cannot safely serve requests.

4.3 React and TypeScript frontend

- Use TypeScript strict mode and typed API request/response models. Avoid `any`; if unavoidable, isolate it and document why.
- Keep API calls in a shared client/service layer rather than scattering raw fetch calls throughout components.
- Represent loading, empty, success, validation-error, unauthorized, and server-error states explicitly.
- Do not rely on frontend validation or hidden buttons for security; the backend is the enforcement boundary.
- Do not store long-lived secrets or merchant API keys in browser code. Avoid persisting sensitive tokens in local storage; follow the team's approved session/token strategy.
- Use accessible labels, keyboard-operable controls, meaningful error text, and responsive layouts.
- Never render untrusted HTML directly. Escape or sanitize user-controlled content when rich text is truly required.

4.4 Python AI service

- Use type hints, input/output schemas, explicit timeouts, bounded retries, and structured logging.
- Keep tool functions small and narrowly scoped. The model may select from approved tools but must not gain arbitrary database, shell, or unrestricted network access.
- Validate tool arguments independently of model output. Treat model-generated arguments and user-provided content as untrusted.
- Return safe, honest fallback responses when tools or the model fail; never fabricate parcel status, fees, or delivery events.
- Keep provider API keys in secrets/environment configuration, not source code or frontend bundles.

5. API design and integration standards

5.1 REST conventions

- Use the agreed versioned base path (recommended `/api/v1`) and consistent plural resource names.
- Use HTTP methods consistently: GET reads, POST creates/actions, PUT replaces where appropriate, PATCH partially updates, DELETE removes/deactivates where appropriate.
- Use appropriate status codes: 200/201 for success, 204 for successful no-content responses, 400 for malformed input, 401 for unauthenticated, 403 for forbidden, 404 for not found, 409 for conflicts, 422 for semantic validation if adopted, and 429 for rate limits.
- Publish OpenAPI documentation or an equivalent contract for each public/internal API used by another member.
- Use DTOs and explicit response fields; do not serialize database rows directly by default.
- Use pagination for potentially large lists and document sorting/filtering rules.
- Use a consistent error envelope, for example an error code, human-readable message, request/correlation ID, and field-level validation details where appropriate.
- Use UTC timestamps in ISO 8601/RFC 3339 form. Define currency, weight, size, and distance units explicitly.

5.2 Validation and idempotency

- Validate required fields, formats, lengths, ranges, enum values, and cross-field constraints on the server.
- Use idempotency keys for operations where retries can create duplicates, especially parcel booking and merchant order creation.
- Define uniqueness constraints in the database as well as application validation where uniqueness matters.
- Do not treat client-supplied price or role as authoritative. Calculate fees and authorize roles on the server.
- Do not expose different sensitive error details that allow attackers to enumerate accounts or secrets.

5.3 API security and compatibility

- Apply authentication and authorization consistently to every protected endpoint.
- Enforce object ownership and merchant tenant isolation in backend queries and service logic.
- Use request size limits, rate limiting for sensitive/public endpoints, CORS allowlists, and reasonable timeouts.
- Use correlation IDs across HTTP requests, outbox events, worker logs, and webhook attempts.
- Do not change an API or event schema incompatibly without coordination, tests, and a versioning/migration plan.

6. Database and data management standards

- Use PostgreSQL as the authoritative store for business data. Agree a canonical schema and migration process before parallel feature development.
- Each feature owner owns migrations for their tables; a nominated database coordinator reviews naming, relationships, constraints, indexes, and migration ordering.
- Use primary keys, foreign keys, `NOT NULL`, `CHECK`, and unique constraints to enforce invariants where appropriate.
- Store money as an exact numeric/decimal representation or integer minor units; do not use binary floating point for prices. Agree currency and rounding rules.
- Use transactions for logically atomic updates. Keep transactions short and do not make slow external HTTP calls while holding a database transaction open.
- Add indexes based on query patterns and measured needs, especially tracking-number lookup, parcel history, assignments, outbox polling, and webhook delivery status.
- Use explicit created/updated timestamps and actor/audit fields for important operational changes.
- Store only necessary personal data. Define retention/deletion rules and avoid copying sensitive data into logs, events, or analytics unnecessarily.
- Use migrations for schema changes; do not rely on manually editing a shared database.
- Use separate local/test data and never seed real personal data into the demo environment.

7. Authentication, authorization, and privacy

- Use the agreed application authentication approach (JWT or server-side session) consistently. Validate signatures, issuer/audience where applicable, expiry, and relevant OIDC state/nonce protections.

- Hash passwords with a suitable password-hashing algorithm and library if local passwords are supported; never store plaintext or reversible passwords.
- Define roles for customer, merchant, courier, dispatcher, and admin, but combine role checks with resource ownership and tenant checks.
- Enforce least privilege: services, database users, and credentials should receive only the permissions needed for their task.
- Public tracking must expose only an approved minimal set of fields. Do not return full addresses, phone numbers, private delivery notes, proof photos, or merchant-only data unless specifically authorized.
- Protect API keys and webhook secrets; support revocation/rotation where feasible and never print them in logs.
- Use HTTPS/TLS for deployed environments and secure cookie settings if cookies are used.
- Protect against brute force and enumeration with rate limits, generic auth errors where appropriate, and monitoring.
- Collect only necessary personal data, document why it is needed, restrict access, and agree retention/deletion behavior with the team.
- Do not assume Google sign-in alone authorizes access to a parcel or merchant tenant; application authorization is still required.

8. Messaging, outbox, notifications, and webhooks

- Domain writes must create outbox rows in the same PostgreSQL transaction as the business change they describe.
- The Outbox Relay publishes persistent messages and marks them published only after broker confirmation. A crash may cause duplicate publication; consumers must be idempotent.
- Use durable queues, explicit acknowledgements, bounded retries, exponential backoff with jitter, and a dead-letter queue (DLQ) for exhausted failures.
- Do not acknowledge a message before the consumer's required work has been durably completed.
- Give every event a stable event ID, event type, schema version, timestamp, aggregate/parcel ID, and correlation ID.
- Consumers must tolerate duplicate and out-of-order events where relevant; use event IDs, version checks, or idempotency records.
- Separate transient errors (retry may help) from permanent errors (invalid destination/payload should be investigated or dead-lettered).
- Sign outbound webhook payloads with a documented scheme and timestamp; document receiver verification and replay protection.
- Validate webhook destination URLs and defend against SSRF: block loopback, link-local, private/internal ranges where appropriate, unsafe redirects, and non-HTTP(S) schemes.
- Keep webhook secrets out of payloads and logs. Apply timeouts, response-size limits, and a bounded retry policy.
- Reconciliation should identify and safely recover stale work or raise an alert. It must not bypass the relay, blindly republish forever, or invent parcel state.
- Use a local fake notification provider/webhook receiver for repeatable tests and demos.

9. Tracking, realtime, and cache rules

- Tracking reads must reflect the latest committed authoritative state within the agreed target. Do not let a stale cache or eventual projection silently become the truth.
- Use Server-Sent Events (SSE) for one-way server-to-browser status updates unless the team has a clear reason for WebSockets.
- Define heartbeat, reconnect, event IDs, and polling fallback behavior; disconnects must not permanently lose the latest status.
- Authorize private tracking/history separately from public tracking and apply rate limits to public tracking numbers.
- If multiple Tracking API replicas need live fan-out, Redis Pub/Sub may be used as a notification mechanism; clients should refresh authoritative state after reconnect or missed events.
- Never put sensitive fields into realtime event payloads merely because the browser can hide them.

10. Kubernetes, containers, and configuration

- Each service must have a reproducible container build, documented port, startup command, configuration list, and health endpoints.
- Use Docker Compose for local development and a documented Kubernetes deployment for the bonus. Keep the difference between local and cluster configuration explicit.
- Use ConfigMaps for non-sensitive configuration and Secrets or an approved secret manager for credentials. Commit only placeholder examples.
- Define readiness/liveness probes, graceful shutdown, resource requests/limits, and restart behavior for workloads where applicable.
- Expose services internally by default; use Ingress/API Gateway for intentional external access. Avoid exposing databases, RabbitMQ management interfaces, or internal workers publicly.
- Use Services for stable in-cluster discovery and namespaces/labels consistently.
- Use HPA for appropriate stateless APIs; use queue-depth-based scaling such as KEDA for workers only when metrics and scaling behavior are configured and tested.
- Do not put persistent database/broker state on ephemeral container filesystems. Use suitable persistent storage or document an approved managed-service choice.
- Pin image tags for repeatable demos; avoid relying on `latest` for the final submission.
- Provide a clean-clone setup test and a documented teardown/reset procedure.

11. Observability and operational readiness

- Use structured logs with severity, timestamp, service, request/correlation ID, and relevant non-sensitive identifiers.
- Never log passwords, access tokens, API keys, webhook secrets, full payment/identity secrets, or unnecessary personal data.
- Provide health endpoints that distinguish process liveness from readiness to serve requests.
- Expose metrics for request rate, latency, error count, broker queue/backlog, consumer failures, outbox backlog, webhook outcomes, and dependency failures where relevant.
- Use consistent metric names and labels; do not use parcel IDs, user IDs, or other high-cardinality values as metric labels.
- Record actionable errors and include enough context to debug without exposing sensitive payloads.
- Document how to inspect logs, metrics, queue/DLQ state, and common recovery actions.
- Set explicit timeout and retry budgets to avoid retry storms and cascading failures.

12. Testing and quality gates

12.1 Required test layers

- Unit tests: business rules, fee calculations, state transitions, validation, authorization helpers, and retry decisions.
- Integration tests: real database/broker dependencies or controlled test instances for migrations, transactions, outbox publication, consumers, and API persistence.
- API/security tests: missing/invalid/expired credentials, role denial, object ownership, merchant tenant isolation, malformed input, and rate limits where practical.
- End-to-end tests: customer booking → tracking; dispatcher assignment → courier update → delivered/failed; merchant order → status webhook; notification/retry/DLQ path.
- Frontend tests: core user journeys, loading/error states, form validation, and role-specific views.
- AI tests: tool argument validation, permissions, timeout/provider failure, prompt-injection boundaries, and no fabricated operational facts.

12.2 Test quality rules

- Tests must be repeatable and must not depend on real customers, personal data, paid notification delivery, or uncontrolled external services.
- Use deterministic fixtures and unique test data. Clean up resources or isolate test databases/queues.
- Test negative and failure paths, not only successful requests.
- Do not make tests pass by swallowing errors or weakening assertions.

- Record coverage honestly. Coverage percentage is useful, but critical business/security paths matter more than chasing a number.
- Keep a test report mapping major requirements to test cases and results.

12.3 Merge gates

- Before merge: format, lint, compile/build, relevant unit tests, and secret scanning or a manual secrets check.
- Before release/demo: migration from a clean database, full local stack startup, core end-to-end scenarios, authorization tests, and a clean-clone setup check.

13. AI assistant and MCP safety

- Baseline tools should be read-only: track a parcel, retrieve authorized delivery history, and estimate a fee.
- Expose tools with narrow schemas and explicit descriptions; validate every argument server-side.
- The assistant must call the backend's approved API, not access PostgreSQL directly or infer current parcel facts from model knowledge.
- Apply the same user/tenant permissions to tool calls as to direct API calls. A tool call is not a privilege escalation.
- Treat user messages, parcel notes, merchant content, and tool output as untrusted data; they must not override system/developer policies or tool allowlists.
- Do not allow arbitrary URLs, SQL, shell commands, or unrestricted network tools.
- Limit tool call count, output size, latency, and retries. Redact private fields and do not return hidden internal records.
- If a tool fails or returns no record, say so plainly; do not guess a parcel status or fee.
- Log tool name, outcome, latency, and correlation ID without storing unnecessary prompts or sensitive payloads.

14. Documentation requirements

- Root README: project purpose, architecture diagram, prerequisites, clean-clone setup, Compose startup, migrations, seed data, tests, and troubleshooting.
- Co-located Living Module Documentation (README.md): Mandatory for every internal module in core-api/internal/ and every independent service. Must be updated synchronously on every code change to detail full ownership, domain invariants, entities, layer organization, database tables, and current status.
- Co-located Living API Documentation (API.md): Mandatory for every module and service exposing HTTP endpoints. Must specify complete request headers, query parameters, request body schemas, response envelopes, HTTP status codes, and error payloads; must be updated concurrently with every endpoint change.
- Event contract: event names, schema versions, examples, delivery guarantees, idempotency, retry/DLQ behavior, and webhook signatures.
- Architecture diagrams: system/container view, deployment view, database/ER view, and key sequence flows.
- Operational guide: deployment commands, health checks, logs/metrics, queue/DLQ investigation, safe replay, backup/reset assumptions, and rollback notes.
- Decision log: record important choices, date, options considered, decision, rationale, and impact.
- Contribution record: each member's completed work, tests, documentation, and demo explanation.

15. Definition of done and exception process

- A feature is done only when implementation, tests, authorization, error paths, documentation, and integration are complete enough to meet its acceptance criteria.
- An exception to a MUST/SHOULD rule must be recorded with the reason, risk, owner, mitigation, and date to revisit.
- Do not mark a requirement complete just because a screen exists; verify the backend behavior, persistence, permissions, and failure handling.
- Keep the main branch runnable and integrate incrementally. A feature that only runs on one member's machine is not complete.